Academic Year	Module	Assessment Number	Assessment Type
2026	Artificial Intelligence and Machine Learning(6CS012)	3	Report

Financial Sentiment Classification using RNN, LSTM and Word2Vec

Student Id: 2431235

Student Name: Sujal Shrestha

University Email: s.shrestha202@wlv.ac.uk

Section: L6CG19

Tutor: Bijaya Ghimire

Abstract

This project is about Financial sentiment classification which aims at utilizing deep learning models. The main goal of the project is to categorize financial sentences into positive, negative and neutral sentiment categories. Businesses and investors are interested in sentiment analysis because it enables them to automatically analyse opinions and financial trends from text data. Three deep learning models were implemented in this project: Simple RNN, LSTM, and LSTM with pretrained Word2Vec embeddings. The dataset was pre - processed by lowercasing, removing stop words, handling contractions and lemmatization. Text data to train was prepared by using keras tokenizer and percentile based sequence padding. The models were tested by measuring their accuracy, confusion matrix, classification report, and training time comparison. By using the pretrained embeddings, the Word2Vec-LSTM model performed better than the Simple RNN model as shown in experimental results, due to the fact that the pretrained embeddings have richer semantic meaning for words. The real-time sentiment prediction was also realized using a Gradio GUI.

Table of Contents

Table Of Figure

1. Introduction

Text classification is one of the most crucial tasks in Natural Language Processing (NLP) (Bird, Klein, & Loper, 2009). In financial systems, sentiment analysis is frequently applied to detect opinions and feelings within textual data, including in social media platforms and for customer feedback analysis. The benefit of financial sentiment analysis is that it can be used to gauge the general sentiment of investors and companies about the company's performance, profitability, or market conditions.

Traditional ML methods need to manually extract the features from the text data while deep learning models like RNN and LSTM can learn patterns from sequential text data automatically. Recurrent Neural Networks are intended to learn sequences, but can have vanishing gradient problems with long text sequences. The memory cells in Long Short-Term Memory (LSTM) networks are designed to store long-term memory, which addresses this issue. The project includes pre-processing, model evaluation, error analysis, and a gradio interface for real time prediction. This project compares three models:

- Simple RNN with trainable embedding
- LSTM with trainable embedding
- Word2Vec-LSTM with pretrained embeddings

2. Dataset Description and preparation

2.1 Dataset Overview

For this project, a dataset with financial sentences with positive, neutral, and negative sentiment labels. The data consists of financial reports, company performance statements and market related news. Pandas was used to load the data and prior to model training the data was cleaned.

```
DATA_PATH = "/content/drive/MyDrive/Ai Assessment/financial_phrase (1).csv"

data = pd.read_csv(DATA_PATH)

print('Dataset shape:', data.shape)
display(data.head(8))
print('Columns:', list(data.columns))
```

```
Dataset shape: (2264, 2)
```

	text	label
0	According to Gran , the company has no plans t...	neutral
1	For the last quarter of 2010 , Componenta 's n...	positive
2	In the third quarter of 2010 , net sales incre...	positive
3	Operating profit rose to EUR 13.1 mn from EUR ...	positive
4	Operating profit totalled EUR 21.1 mn , up fro...	positive
5	Finnish Talentum reports its operating profit ...	positive
6	Clothing retail chain Sepp+ñl+ñ 's sales incre...	positive
7	Consolidated net sales increased 16 % to reach...	positive

```
Columns: ['text', 'label']
```

Figure 1: Preview of the financial sentiment dataset showing sample records and sentiment labels.

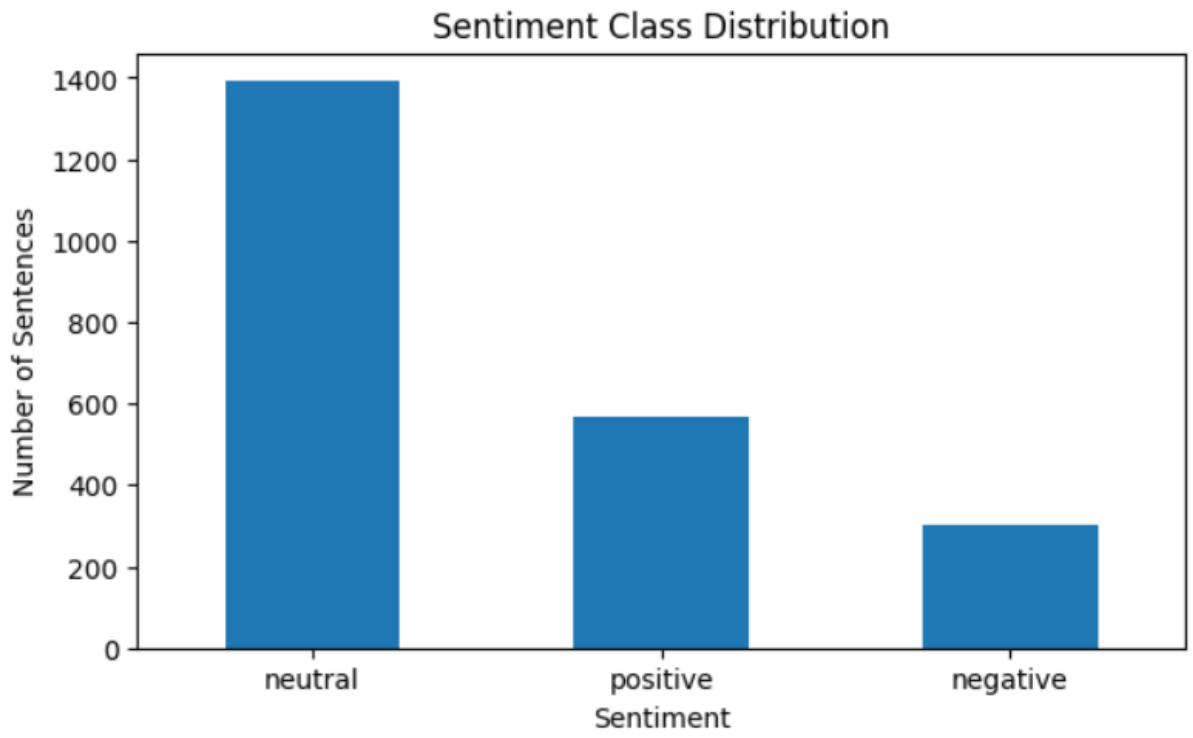


Figure 2Distribution of sentiment classes in the dataset.

Figure 2:

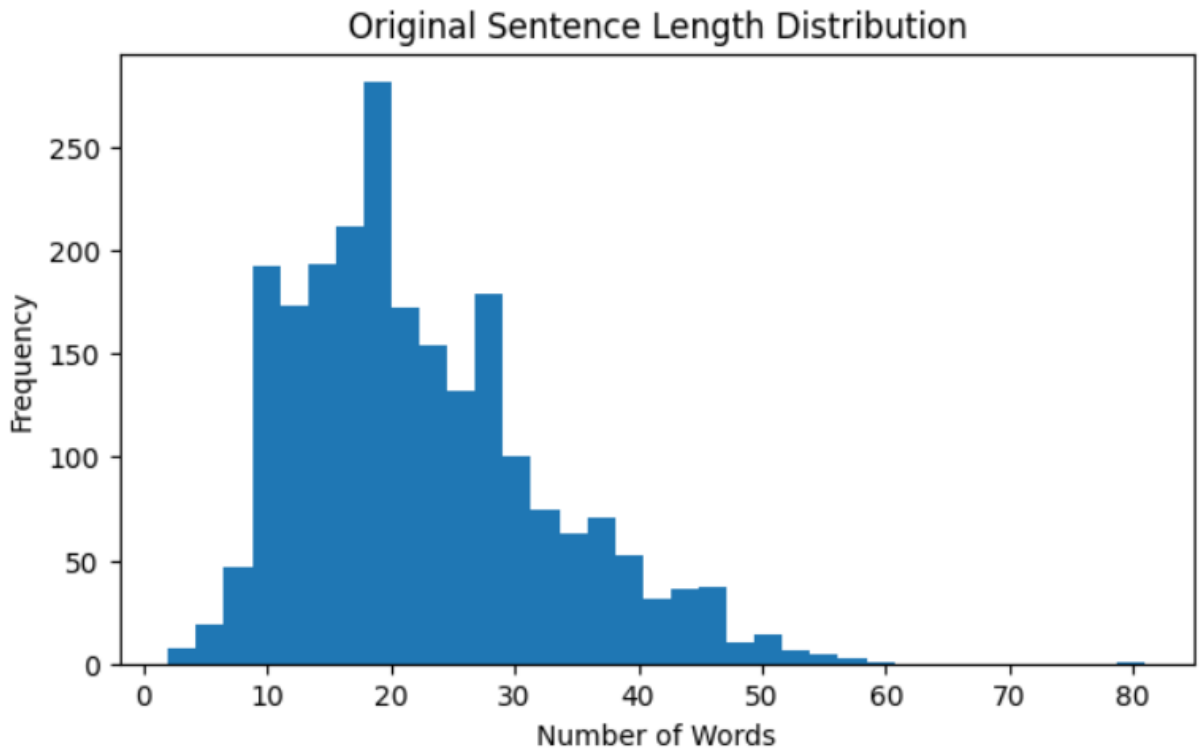


Figure 3: Distribution of sentence lengths in the dataset.

Figure 3

2.2 Text Preprocessing

The text was subsequently pre-processed using several techniques for the purpose of enhancing the quality of the text. All the text was lowercase to ensure uniformity. The reasons for removing URLs, hashtags, mentions, special characters and numbers from the data is that these don't add much to the understanding of sentiment. The contractions were expanded to maintain the meaning of the words, e.g., "don't" was expanded to "do not. Stop words were omitted, except for certain terms that carry the sentiment in the text, such as “not” and “no”, and lemmatization was applied to get words to their base forms (Bird, Klein, & Loper, 2009).

	text	clean_text	label
0	According to Gran , the company has no plans t...	accord gran company no plan move production ru...	neutral
1	For the last quarter of 2010 , Componenta 's n...	last quarter componenta net sales double eur e...	positive
2	In the third quarter of 2010 , net sales incre...	third quarter net sales increase eur mn operat...	positive
3	Operating profit rose to EUR 13.1 mn from EUR ...	operate profit rise eur mn eur mn correspond p...	positive
4	Operating profit totalled EUR 21.1 mn , up fro...	operate profit total eur mn up eur mn represen...	positive
5	Finnish Talentum reports its operating profit ...	finnish talentum report operate profit increas...	positive
6	Clothing retail chain Sepp+ñil+ñ 's sales incre...	clothe retail chain sepp l sales increase eur ...	positive
7	Consolidated net sales increased 16 % to reach...	consolidate net sales increase reach eur opera...	positive
8	Foundries division reports its sales increased...	foundries division report sales increase eur m...	positive
9	HELSINKI (AFX) - Shares closed higher , led ...	helsinki afx share close higher lead nokia ann...	positive

Shape after text cleaning: (2264, 4)

Figure 4: Example of text after pre-processing and cleaning

Figure 4.

Figure 5: Word cloud generated from cleaned financial text.

Figure 5

2.3 Tokenization and Sequence Padding

Cleaning was followed by tokenization with the Keras Tokenizer where text was transformed into sequences of integers (Chollet, 2021). The sequences were padded with percentiles for uniformity without adding too many. The data set was then split into test and training sets with the ratio of 80% training data and 20% testing data.

```
Classes: ['negative', 'neutral', 'positive']
Encoded label example: [1 2 2 2 2 2 2 2 2 2]
Vocabulary size: 3993
Maximum sequence length: 23
Training data shape: (1811, 23)
Testing data shape: (453, 23)
```

Figure 6: Tokenization and sequence padding results

3. Model Architectures

3.1 Simple RNN

The project deployed three deep learning architectures for financial sentiment classification. The project tested three deep learning architectures for financial sentiment classification: The first model was a trainable embedding layer Simple RNN. This model was trained directly with the data to learn word representations. Since the dataset has several sentiment classes, the simple RNN models were compiled using the Adam optimizer and sparse categorical cross entropy loss. The simple RNN architecture consists of Embedding layer, RNN layer, Dropout layer, Dense output layer.

Model: "Student_Simple_RNN"

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 23, 50)	199,650
simple_rnn (SimpleRNN)	(None, 50)	5,050
dropout (Dropout)	(None, 50)	0
dense (Dense)	(None, 32)	1,632
dense_1 (Dense)	(None, 3)	99

Total params: 206,431 (806.37 KB)
Trainable params: 206,431 (806.37 KB)
Non-trainable params: 0 (0.00 B)

Figure 7: Simple RNN architecture summary.

3.2 Long Short-Term Memory (LSTM)

The second model used Long Short-Term Memory (LSTM) layer instead of recurrent layer, which was used to learn long-term dependency and to minimize vanishing gradient problems. The LSTM model included Embedding layer, LSTM layer, Dropout layer, Dense layer, Softmax output layer. Compared to Simple RNNs, LSTMs generally achieve better performance on text classification tasks because they handle long-range contextual information more effectively .

Model: "Student_LSTM_Model"

Layer (type)	Output Shape	Param #
embedding_1 (Embedding)	(None, 23, 50)	199,650
lstm (LSTM)	(None, 64)	29,440
dropout_1 (Dropout)	(None, 64)	0
dense_2 (Dense)	(None, 40)	2,600
dense_3 (Dense)	(None, 3)	123

Total params: 231,813 (905.52 KB)
Trainable params: 231,813 (905.52 KB)
Non-trainable params: 0 (0.00 B)

Figure 8: LSTM architecture summary

3.3 Pretrained Word2vec-Lstm Model

The third model is based on pretrained Word2Vec embeddings and an LSTM layer. Word2Vec embeddings generate a semantic vector to represent a word based on a huge external corpus, which better understands the relationship between financial terms (Mikolov et al., 2013). In general, pretrained embeddings help to improve performance since the model is not starting from scratch but rather from representations of words with meaning.

Model: "Student_Word2Vec_LSTM"

Layer (type)	Output Shape	Param #
embedding_2 (Embedding)	(None, 23, 100)	399,300
bidirectional (Bidirectional)	(None, 96)	57,216
dropout_2 (Dropout)	(None, 96)	0
dense_4 (Dense)	(None, 36)	3,492
dense_5 (Dense)	(None, 3)	111

Total params: 460,119 (1.76 MB)
Trainable params: 60,819 (237.57 KB)
Non-trainable params: 399,300 (1.52 MB)

Figure 9: Word2Vec-LSTM architecture summary

4. Training Strategy and Evaluation

4.1 Training Configuration

Since all the dataset has several sentiment classes, all of them were compiled using the Adam optimizer and sparse categorical cross entropy loss (Chollet, 2021). To avoid overfitting and restore the best model weights automatically while training, the early stopping call backs were also added.

4.2 Evaluation Metrics

Accuracy, Precision Recall, F1-score, Confusion Matrix, Classification Report evaluation metrics were used . The main criterion for evaluation was accuracy. Weighted averages were used to account for class imbalance within the dataset.

5. Experiment and Results and Comparisons

For financial sentiment classification, the project has adopted 3 deep learning architectures. The first model was a Simple RNN where the embedding layer was trainable. In this model, word representations were learned directly from the training data. The second model takes into account the long-term dependency by adding an LSTM layer instead of the recurrent layer and mitigates the vanishing gradient problem. The third model was based on pretrained word2vec embeddings and an LSTM layer. Word2Vec embeddings generate a vector representation of the word from a large external corpus so that the relationships between financial terms are better understood. Pretrained embeddings typically boost results since the model is provided with useful word representations, rather than having to construct them from scratch.

With the data set containing several categories of sentiment, all models have been compiled with the Adam optimizer and sparse categorical cross-entropy loss function. The primary criteria for evaluation was accuracy. In addition, early stopping callback and automatic restore best model weights were added to avoid overfitting and to restore the best model weights during the training (Mikolov et al., 2013).

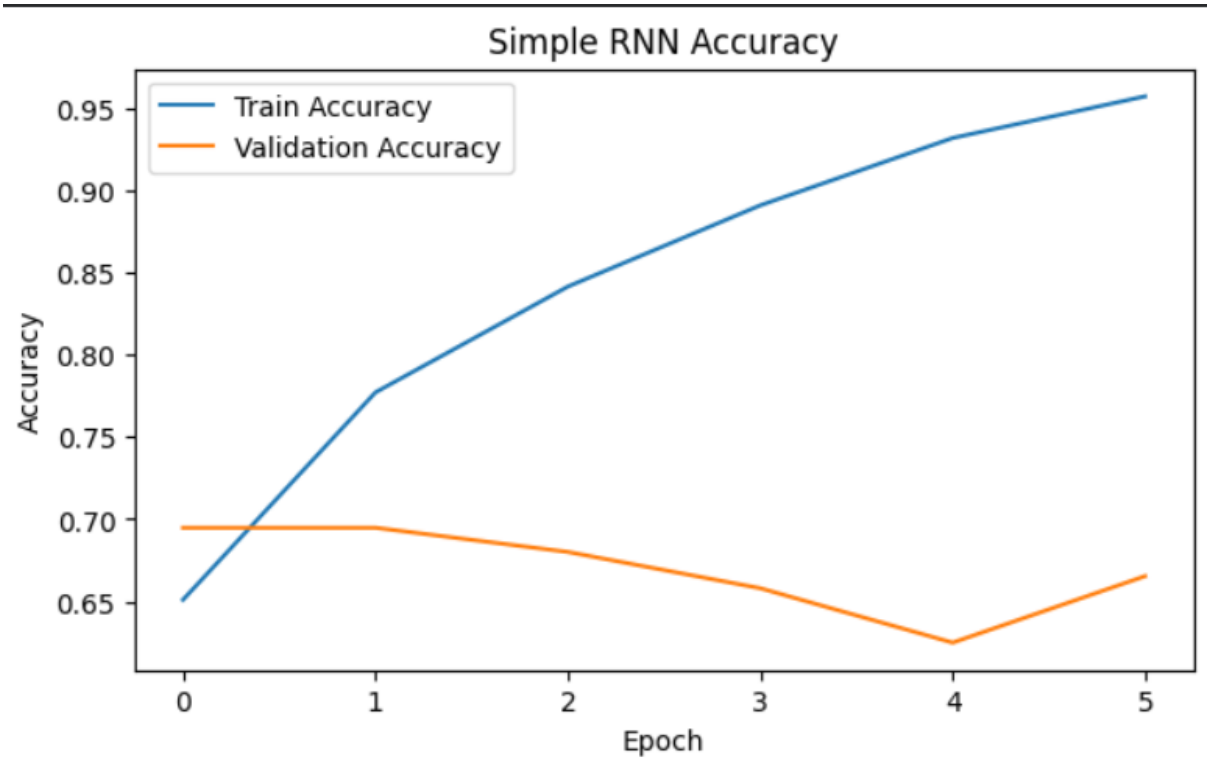


Figure 10: Training and validation performance of the Simple RNN model.

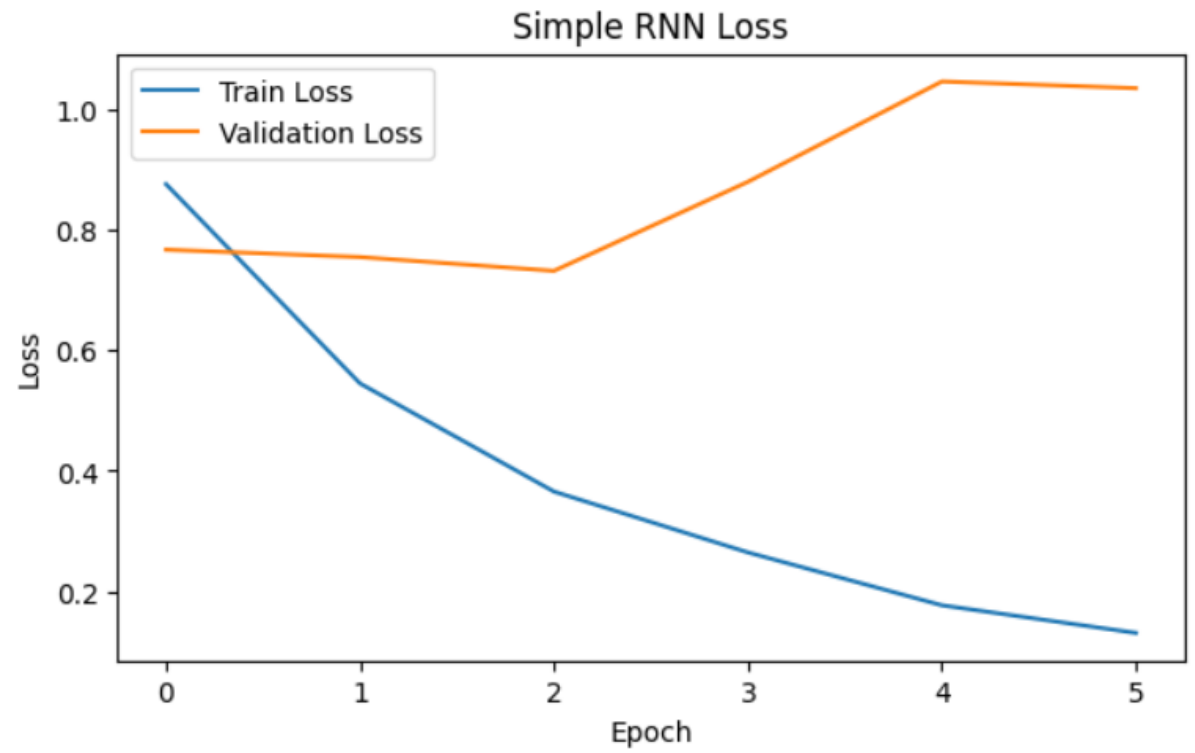


Figure 11: Training and validation performance of the Simple RNN model.

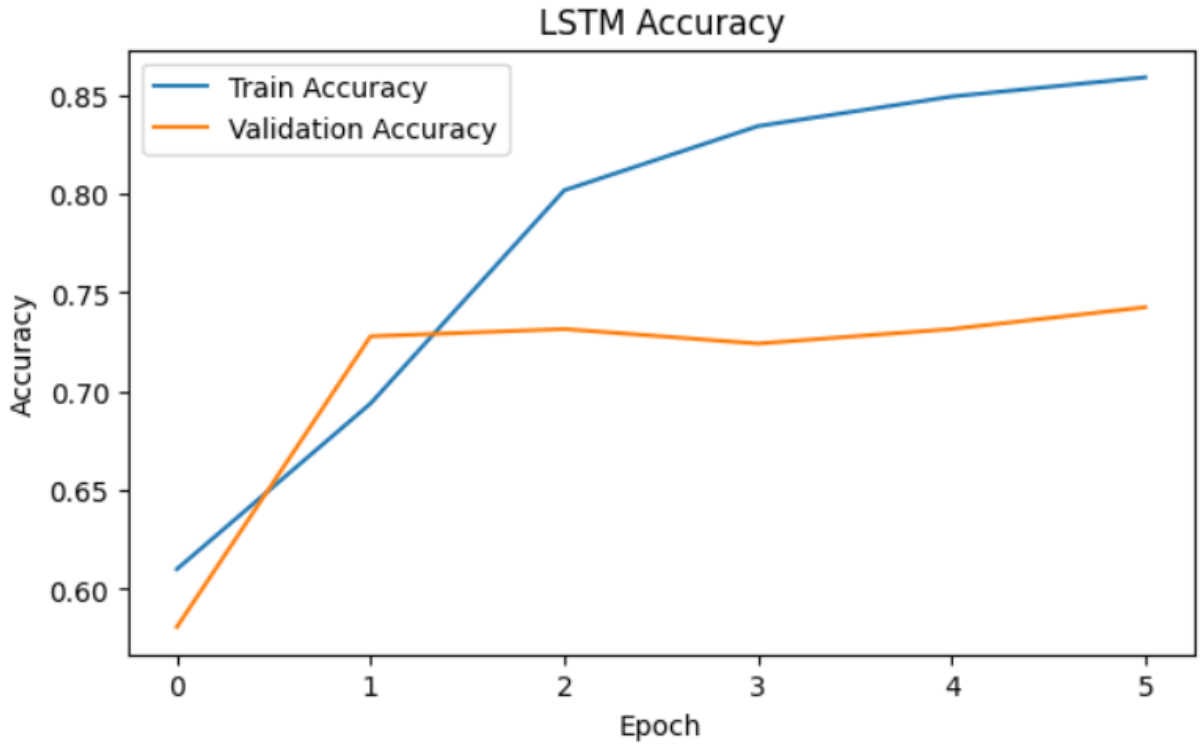


Figure 12: Training and validation performance of the LSTM model.

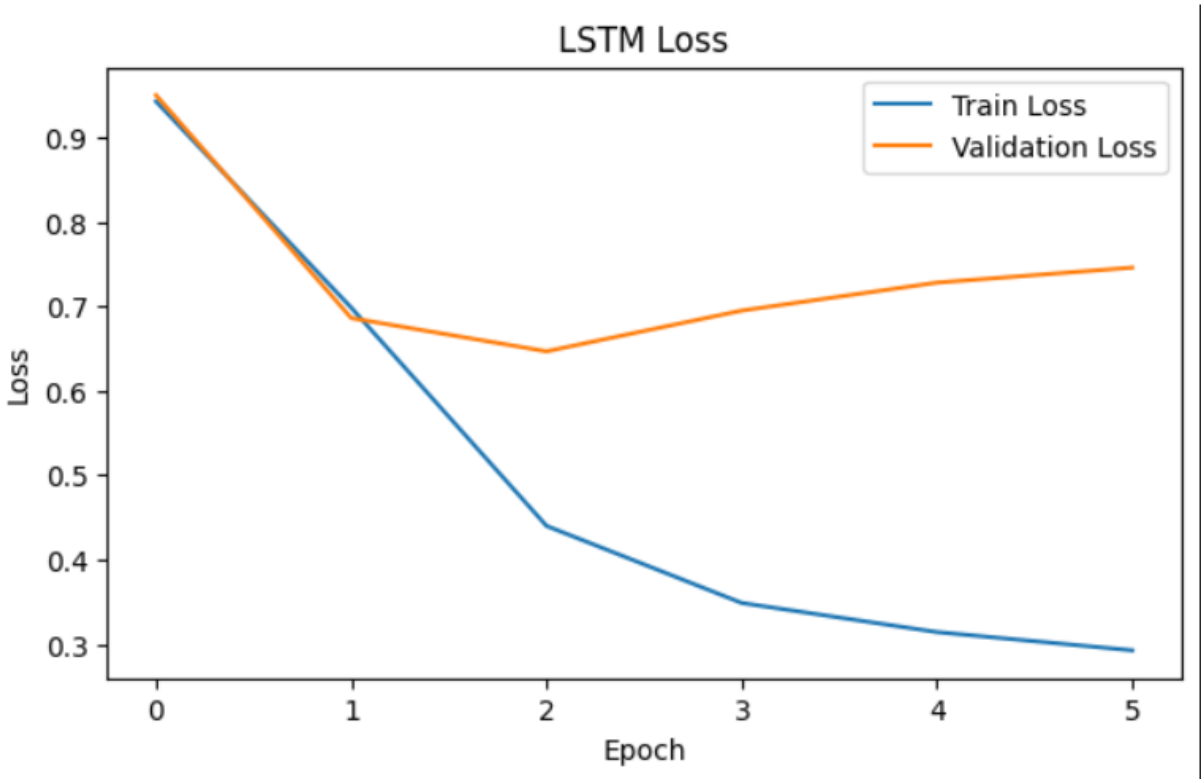


Figure 13: Training and validation performance of the LSTM model.

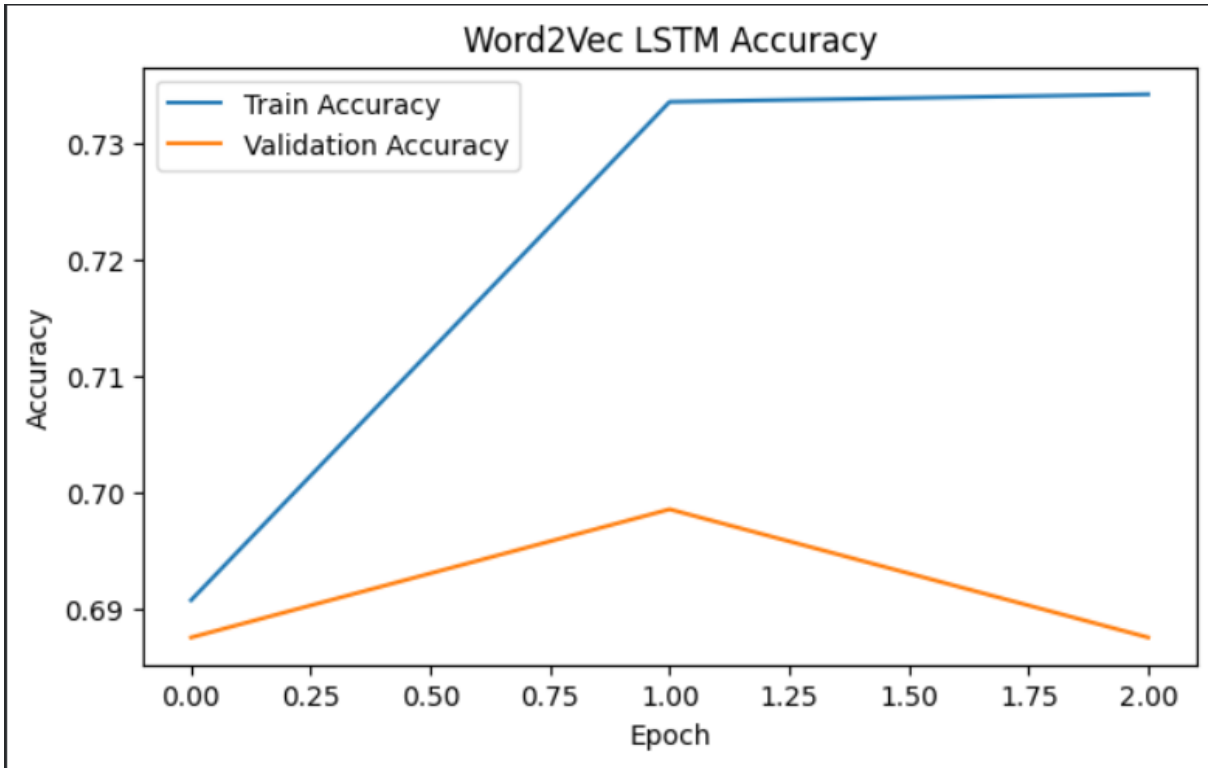


Figure 14Training and validation performance of the Word2Vec-LSTM model.

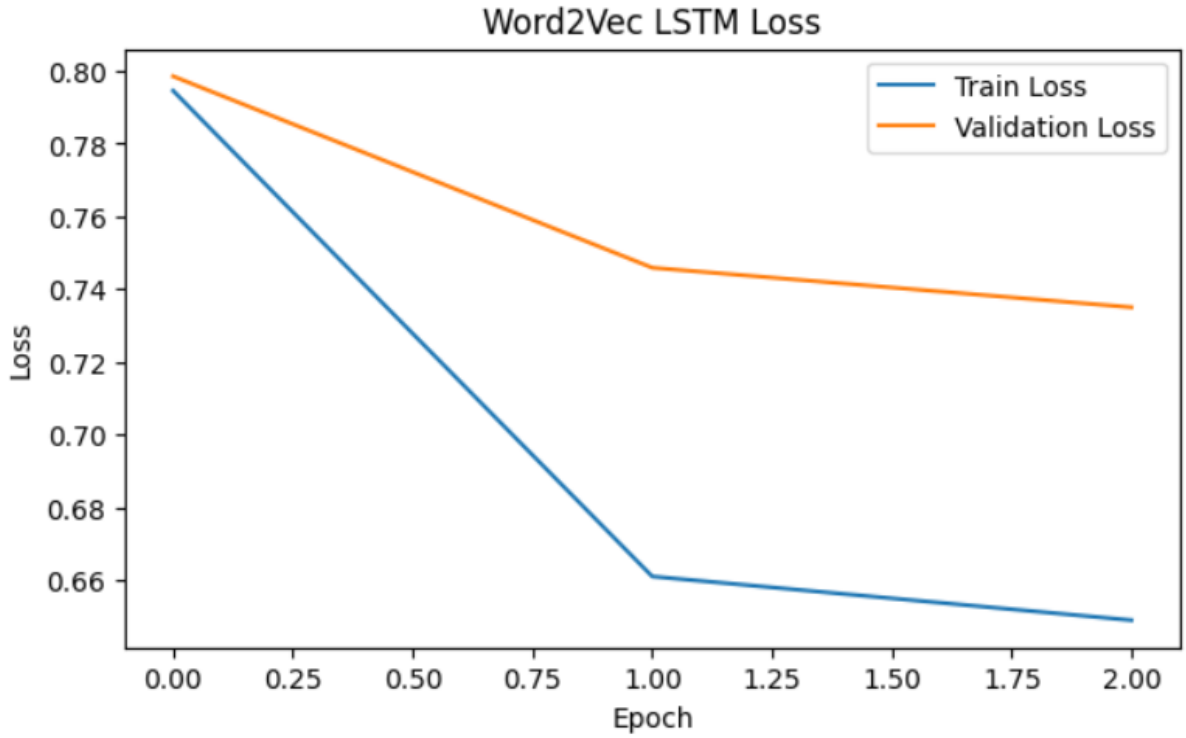


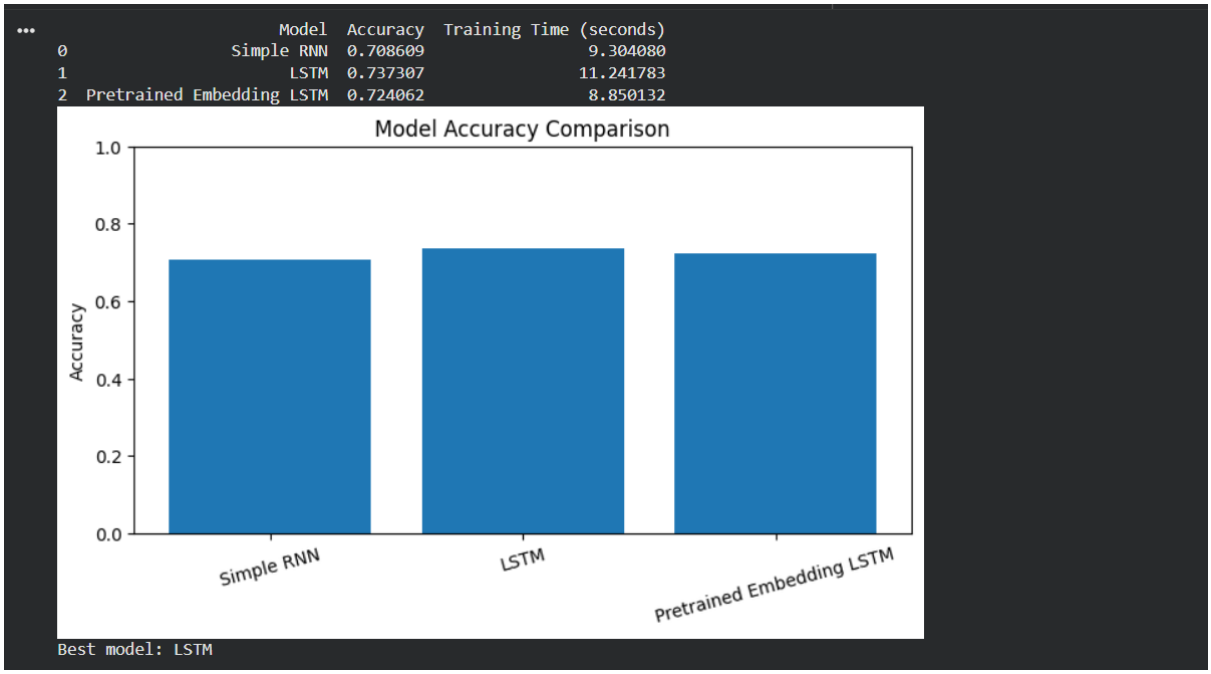
Figure 15Training and validation performance of the Word2Vec-LSTM model.

6. Confusion Matrix and Error Analysis

Even though the models performed well in terms of classification, there were a few false predictions during testing. There were some financial sentences that were short and contextually limited, which made sentiment classification challenging. Furthermore, neutral financial statements may be accompanied by words that are associated with positive and/or negative sentiment, causing models to mix up.

The class imbalance was another obstacle. The data set had more neutral samples than negative, which led to a slight bias of the model to predict neutral sentiment. In addition, the embeddings might not have been trained with sufficient financial knowledge or on rare financial expressions. Some improvements for future work include:

- Using larger financial datasets
- Applying transformer-based models such as BERT
- Using financial-domain pretrained embeddings
- Improving dataset balancing techniques




```
                        Model  Accuracy  Training Time (seconds)
0                  Simple RNN  0.708609                9.304080
1                        LSTM  0.737307               11.241783
2  Pretrained Embedding LSTM  0.724062                8.850132
```

```
Best model: LSTM
```

Figure 16Comparison of model accuracy and training time.

7. Real Time Prediction interface

For better usability, a gradio graphical user interface (GUI) was implemented to perform real-time sentiment prediction. Financial text can be typed into the interface, and the trained model makes a prediction of whether the sentence is positive, negative or neutral. This illustrates the application of the trained sentiment classification system in real-life situations.

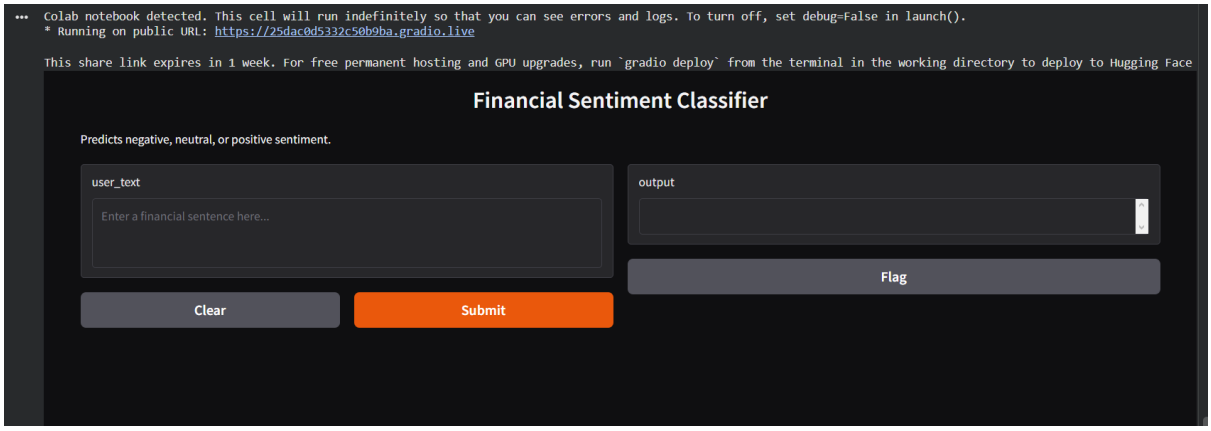


Figure 17: Gradio-based real-time financial sentiment prediction interface.

8. Conclusion

The financial sentiment classification was successfully performed using Simple RNN, LSTM and pretrained Word2Vec-LSTM models in this project. The data was pre-processed for deep learning using text pre-processing, tokenization, and sequence padding methods. The experimental results demonstrated that the pretrained Word2Vec-LSTM model outperformed the other models, as the pretrained embeddings helped the model to better grasp the meaning of financial texts.

The project also highlighted the significance of deep learning models for Natural Language Processing (NLP) related applications like sentiment analysis. The problems identified through the error analysis were the short and ambiguous financial sentences and class imbalance. Overall, the project is a solid basis for further research in the field of NLP and offers a glimpse into the potential of sentiment analysis systems for financial decision-making applications.

9. References

1. Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). *Efficient Estimation of Word Representations in Vector Space*. arXiv preprint arXiv:1301.3781.
2. Chollet, F. (2021). *Deep Learning with Python* (2nd ed.). Manning Publications.
3. Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3rd ed.). O'Reilly Media.
4. TensorFlow Developers. (2026). *TensorFlow Documentation*. Retrieved from https://www.tensorflow.org/
5. Keras Developers. (2026). *Keras Documentation*. Retrieved from https://keras.io/
6. Gradio Developers. (2026). *Gradio Documentation*. Retrieved from https://www.gradio.app/